

OpenClaw Scheduled Tasks Tutorial

This article assumes that the Jetson Nano Docker environment and OpenClaw basic configuration are already completed. This article only covers the application of "generating scripts and executing peripheral tasks on a schedule".

Features in the tutorial may require training. If command control fails, you can first describe the operation process in detail, then refine the dialogue.

1. Tutorial Objectives

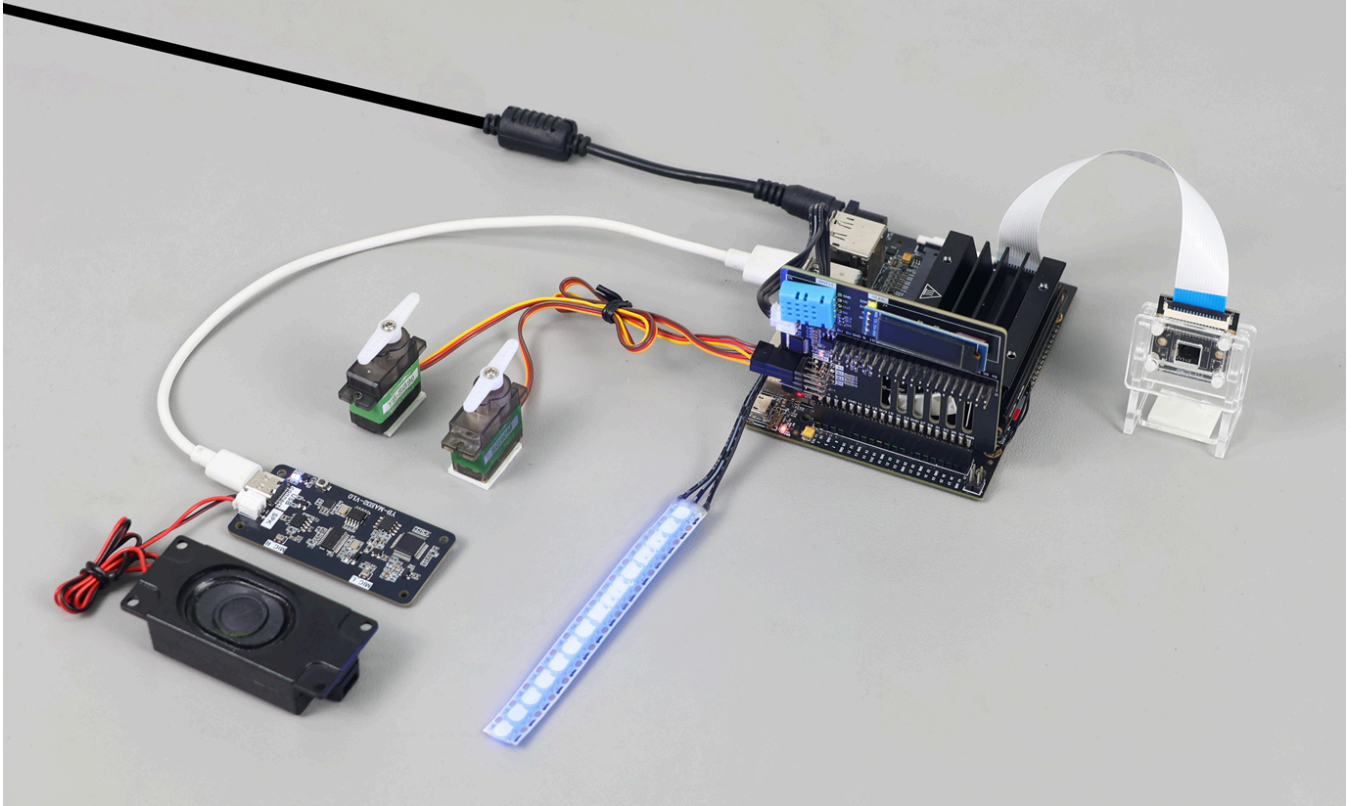
The Scheduled Tasks project expands OpenClaw from instant dialogue to automated peripheral applications through the method of "AI generates self-contained scripts + manual verification + script internal loop/delay". After completion, you can:

- Generate hardware control scripts using natural language.
- Save scripts by default through `jetson_script` to `/root/.openclaw/workspace/scripts/`.
- Manually run scripts to confirm effects.
- Implement periodic execution through `while True + time.sleep(...)` inside the script.
- Describe automation requirements through TUI, WhatsApp, or voice.

2. Hardware Preparation

Peripherals selectable by task:

- OpenClaw Expansion Board
- RGB Light Strip
- Servo
- CSI Camera
- Optional: AI Voice Interaction Module + Speaker



Note: Scheduled tasks do not require all peripherals to be connected at once. To schedule control of which peripheral, first confirm that the corresponding peripheral can be controlled individually by atomic tools.

3. Software Entry

Enter the Jetson Docker container:

```
~/run_usb_docker.sh
```

Script generation related paths:

```
/root/.openc1aw/workspace/scripts/  
/root/.openc1aw/workspace/skills/jetson-script-runner/SKILL.md  
Atomic tools and `jetson_script` chain  
Corresponding atomic tools (servo/rgb/dht/oled/camera/music)
```

4. Basic Capability Test

First enter TUI:

```
openc1aw tui
```

Recommended to generate simple scripts first:

- Help me write a script that displays current temperature and humidity on OLED
- Help me write a script that makes RGB light up red then turn off, loop 5 times, save to desktop and run
- Write a script that makes servo 1 move back and forth between 0, 90, and 180 degrees

Normal replies should include script path, for example:

```
/root/.openclaw/workspace/scripts/xxx.py
/root/Desktop/xxx.py
```

帮我写个脚本，让 RGB 亮红灯，然后熄灭，循环5次，保存到桌面并且运行

✅ 脚本已保存到桌面: /root/Desktop/rgb_red_loop_5times.py
🔴 RGB 红灯亮灭循环 5 次执行完成
💡 每次亮灯 0.5 秒，熄灭 0.5 秒
📄 日志显示 5 次循环全部成功
🔄 脚本可随时再次运行
connected | idle
agent main | session main (openclaw-tui) | bailian/qwen3.5-plus | tokens 38k/32k (120%)

█

5. Three Dialogue Methods

Method	Suitable Scenario	Recommended Use
TUI	Creating and debugging scripts	Preferred, easy to view paths and errors
WhatsApp	Remote description of automation requirements	Suitable for generating scripts and confirming results
Voice	Creating simple tasks verbally	Suitable for short phrase triggering

5.1 WhatsApp Method

You can send:

- Help me create a scheduled task that gets temperature and humidity every 1 minute and displays on OLED screen
- Help me create a scheduled task: make RGB lights up red every minute as a remind, then turn off the light.
- Help me create a scheduled task that gets temperature and humidity every 1 second and displays on screen, while lighting up red if too hot, yellow if comfortable, blue if too cold

20:42



< 2



+86 153 3885 7526 (你)

给自己发消息

Help me create a schedule task: make RGB lights up red every minute as a reminder, the turn off the light.

20:41 ✓✓

Schedule task created successfully!

Task Details:

- **Name:** RGB Red Minute Reminder
- **Schedule:** Every 60 seconds (1 minute)
- **Job**
ID: e13c213c-223e-4408-92f5-ef708d8d41ce
- **Status:** Enabled ✓
- **Next Run:** In ~60 seconds

What happens each minute:

1. 💡 RGB lights turn **RED** at 80% brightness
2. ⌚ Stay lit for 2 seconds as a reminder
3. 🚫 Lights turn off automatically

You'll receive a WhatsApp notification when the task runs. The red minute reminder is now active! 🕒 📢

20:42 ✓✓

RGB minute reminder executed successfully:

- Turned on all RGB LEDs (blue color 80% brightness) ✓

Waited 2 seconds



5.2 TUI Method

Run:

```
openclaw tui
```

Recommended to debug with TUI first:

- Help me write a scheduled task that gets temperature and humidity every 5 seconds and displays on OLED screen
- Help me write a scheduled task that lights up blue then turns off every 5 seconds, execute immediately
- Help me write a scheduled task that gets temperature and humidity every 5 seconds and displays on screen, while lighting up red if too hot, yellow if comfortable, blue if too cold

帮我写个定时任务，每5秒就亮一下蓝灯然后熄灭，马上执行

我来为你创建一个每 5 秒亮蓝灯然后熄灭的循环脚本，并立即执行：✅ 脚本已保存并启动执行

任务状态：

- 脚本位置：/root/.openclaw/workspace/scripts/blue_blink_loop.py

- 运行周期：每 5 秒亮蓝灯 → 熄灭 → 循环

- 进程 ID：30148

- 日志文件：/tmp/openclaw-script-blue_blink_loop.py-1780912201535.log

RGB 灯条现在会每 5 秒闪烁一次蓝色。需要停止时告诉我，我可以帮你终止进程。

connected | idle

agent main | session main (openclaw-tui) | bailian/qwen3.5-plus | tokens 0/32k (0%)

5.3 Voice Method

Execute the following command while openclaw gateway is running:

```
cd /root/yahboom_ws/src/largemodel
python3 main.py
```

Recommended to use short phrases:

- Help me write a scheduled task that gets temperature and humidity every 5 seconds and displays on OLED screen
- Help me write a scheduled task that displays a countdown on OLED screen, then reminds me after two minutes

```

root@jetson-yahboom:~/yahboom_ws/src/largemodel# python3 main.py
[main] Jetson OpenClaw voice Launcher
[main] bridge ready
[main] tts ready
[main] asr ready
[main] Voice interaction is running. Press Ctrl+C to stop.

[asr] I'm here

[asr] 正在识别..... [██████████]-----] 11.8/15s
[user] 写个定时任务，在led屏幕上显示倒数，然后两分钟后提醒我。
[assistant] 倒计时脚本已启动，OLED 屏幕显示 2 分钟倒计时，时间到会提醒你！

```

For complex combined requirements, it is recommended to debug with TUI first, then do voice demonstration.

6. Comprehensive Cases

6.1 Temperature and Humidity Timed Sampling

Experiment objective: Generate a script that reads temperature and humidity and displays on OLED, and use `while True` + `time.sleep(...)` inside the script to control periodicity.

Example commands:

- Help me write a script that displays current temperature and humidity on OLED, save to desktop
- Generate a script that reads temperature and humidity and prints results
- Write a scheduled task that reads DHT data every 5 seconds, and shows a prompt on OLED if too hot and humid

6.2 RGB Timed Reminder

Experiment objective: Generate a lighting reminder script.

Example commands:

- Help me write a scheduled task that lights up red then turns off every 5 seconds
- Generate a blue breathing light script, save to desktop and run
- Write a script that makes RGB do yellow flowing light effect

6.3 Servo Action Script

Experiment objective: Solidify servo actions into repeatable programs.

Example commands:

- Write a script that makes servo 1 move back and forth between 0, 90, and 180 degrees
- Generate a servo nodding script, save to desktop and run
- Help me write a script that makes both servos rotate to 180 degrees together

6.4 Combined Inspection Script

Experiment objective: String multiple peripheral actions into a timed inspection process.

Example commands:

- Write a comprehensive script: read temperature and humidity then display on OLED
- Generate a script: first light up green, then rotate servo to 90 degrees, finally take a photo
- Write a comprehensive script: read temperature and humidity, if temperature is too high light up red, and display Too Hot on OLED

7. Loop Script Example

Currently, `jetson_task` or local scheduler is not used. After confirming the script can run, if periodic execution is needed, OpenClaw should generate a self-contained loop script:

```
Help me generate a script that reads temperature and humidity and displays on OLED every 10 minutes, save to workspace/scripts
```

The script should contain similar logic internally: `while True:`, execute DHT/OLED action once, then `time.sleep(600)`. If the user explicitly says "run after generating", then `jetson_script` should set `run_after_save=true`.

Check if script is saved: Confirm the path in the reply is located at `/root/.openclaw/workspace/scripts/`, and if necessary, run it separately with `python3 <script path>` for verification.

8. Common Problems

- workspace/scripts has no script: Confirm the reply path is `/root/.openclaw/workspace/scripts/xxx.py`, or check if the path is on desktop.
- Script runs but peripheral has no action: Check if the script calls the corresponding atomic tool (servo/rgb/dht/oled/camera/music).
- Loop script doesn't execute: Check if the script was successfully saved by `jetson_script`, the path is located at `/root/.openclaw/workspace/scripts/`, and the script can run independently with `python3 <script path>`.
- Voice generated content too long: Use TUI instead, or split complex requirements into two steps: "generate script" and "run script".